System Verification and Validation Plan for DAIIM

Team 6,
Robert Lam
Nathan Hum
Aidan Lao
Keira Laskoski

November 1, 2026

Revision History

Date	Version	Notes
Oct 26	1.0	Added system tests for authentication and task creation
Oct 27	1.1	Aidan: Added system tests for reputation and rewards, penalties, transparency, data handling, and safety; added non-functional tests, usability survey, test symbolic parameters, and reflection

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 2
- 2.4 Relevant Documentation . . . 3

3 Plan **3**
- 3.1 Verification and Validation Team . . . 3
- 3.2 SRS Verification . . . 3
- 3.3 Design Verification . . . 5
- 3.4 Verification and Validation Plan Verification . . . 5
- 3.5 Implementation Verification . . . 6
- 3.6 Automated Testing and Verification Tools . . . 6
- 3.7 Software Validation . . . 7

4 System Tests **8**
- 4.1 Tests for Functional Requirements . . . 8
  - 4.1.1 Accounts and Authentication . . . 8
  - 4.1.2 Inference Task Creation . . . 13
  - 4.1.3 Provider Registration and Staking . . . 15
  - 4.1.4 Inference Requests . . . 20
  - 4.1.5 Reputation, Rewards, and Penalties . . . 25
  - 4.1.6 Transparency and Data Handling . . . 28
  - 4.1.7 Safety . . . 31
- 4.2 Tests for Nonfunctional Requirements . . . 33
  - 4.2.1 Accuracy, Reliability, and Scalability . . . 33
  - 4.2.2 Usability . . . 34
  - 4.2.3 Security, Privacy, and Safety . . . 36
  - 4.2.4 Maintainability, Portability, and Installability . . . 37
- 4.3 Traceability Between Test Cases and Requirements . . . 39

5 Unit Test Description **39**
- 5.1 Unit Testing Scope . . . 39
- 5.2 Tests for Functional Requirements . . . 39

5.2.1 Module 1 . . . 39
5.2.2 Module 2 . . . 39
5.3 Tests for Nonfunctional Requirements . . . 40
5.3.1 Module ? . . . 40
5.3.2 Module ? . . . 40
5.4 Traceability Between Test Cases and Modules . . . 40

6 Appendix **41**
6.1 Symbolic Parameters . . . 41
6.2 Usability Survey Questions . . . 41

7 Appendix — Generative AI Use Disclosure **43**
7.1 AI Tools Used . . . 43
7.2 How and Where Used . . . 43
7.3 Verification of AI-Generated Content . . . 43

8 Appendix — Reflection **44**

List of Tables

1 Verification and Validation Team Roles . . . 4

List of Figures

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

This document ...

2 General Information

2.1 Summary

The software being tested is DAIIM (Decentralized Artificial Intelligence Inference Marketplace), a decentralized platform that allows users to submit artificial intelligence (AI) inference requests and receive aggregated predictions from multiple independent inference providers. The system enables task requesters to submit inference requests for supported machine learning tasks, while inference providers can register for tasks, contribute predictions, and receive compensation for their participation. The platform uses smart contracts deployed on a public blockchain to manage core marketplace operations, including provider registration, staking, request payments, result aggregation, reputation updates, and reward distribution. The system also includes a web-based user interface for interacting with the marketplace, an off-chain backend for coordinating inference requests and communicating with providers, and a database for storing relevant off-chain information. By combining on-chain and off-chain components, DAIIM aims to provide a transparent and decentralized marketplace that reduces reliance on centralized AI inference providers.

2.2 Objectives

The objective of this Verification and Validation (V&V) plan is to build confidence in the correctness, reliability, and usability of the DAIIM platform by verifying that its implemented components satisfy the requirements defined in the Software Requirements Specification (SRS).

Verification will focus on ensuring that the marketplace correctly handles the submission and validation of inference requests, provider registration and participation, prediction collection, result aggregation, and financial settlement. Particular attention will be given to the interactions between the on-chain smart contracts and off-chain components, ensuring that transactions and marketplace state changes are handled consistently and correctly.

Reliability will also be an important focus of the V&V activities. Testing will examine whether the system can produce aggregated results when

multiple providers participate, including scenarios where some providers are unavailable or submit inaccurate predictions. This will help evaluate whether the aggregation and reputation mechanisms behave as intended under different provider conditions.

Usability validation will focus on whether users can interact with the marketplace without requiring extensive knowledge of blockchain technology or machine learning. This includes evaluating whether users can submit inference requests, authorize transactions, monitor request progress, and understand the resulting predictions and associated information.

The V&V plan will primarily be derived from the Software Requirements Specification (SRS), which defines the functional and non-functional requirements used to construct system-level test cases. The Hazard Analysis will also guide testing priorities by identifying potential failures involving inference reliability, transaction integrity, security, availability, and incentive mechanisms. System architecture and design documentation will further support test development by identifying component interactions, data flows, and interfaces that require verification.

Due to project time and resource constraints, certain verification activities will be outside the scope of this plan. The internal correctness of external dependencies, such as the blockchain network, wallet software, and third-party infrastructure, will not be independently verified. Instead, testing will focus on ensuring that DAIIM integrates with and uses these dependencies correctly. Similarly, the internal implementation and predictive capabilities of individual providers' machine learning models will not be independently validated. Testing will instead focus on the collection and aggregation of provider predictions and the resulting marketplace behaviour.

2.3 Custom Documents

The project includes two approved Custom Documents (CDs):

- **Cognitive Walkthrough:** A cognitive walkthrough will be conducted using a wireframe prototype of the planned application interface, from the perspective of inference requesters. This activity will help identify usability issues and areas of ambiguity in the user workflow. The findings will support validation of whether the interface is understandable and learnable, particularly for users with limited experience in blockchain technology or machine learning.

- **Business Plan:** A business plan will be developed following the completion of a successful proof of concept. It will describe the proposed business model and market strategy for DAIIM, including the potential market need, cost breakdown, competition, tokenomics, and execution plan. Although this document does not directly contribute to software verification, it provides additional context regarding the intended operation and long-term objectives of the marketplace.

2.4 Relevant Documentation

The following documents provide essential background and guidance for the V&V activities:

1. **Software Requirements Specification (SRS):** The SRS defines the functional and non-functional requirements of DAIIM. It serves as the primary reference for developing system-level test cases and determining the expected behaviour of the implemented software.

2. **Hazard Analysis (HA):** The Hazard Analysis identifies potential system-level hazards, their causes, and their consequences. It helps prioritize verification activities toward areas where failures could have significant effects, including inference accuracy and reliability, transaction integrity, provider security, and financial settlement.

?

3 Plan

3.1 Verification and Validation Team

All members, aside from the supervisor, will share the responsibilities of designing the system, writing tests, and developing the system.

3.2 SRS Verification

The SRS document will be verified to ensure requirements are unambiguous, verfiable, and complete.

Team Member	Project Role	VnV Responsibilities
Robert Lam	Team Lead	Schedules reviews, demos a
Nathan Hum	DevOps and Test Tooling Lead	Maintains GitHub Actions,
Aidan Lao	QA and Inspection Lead	Keeps the review checklists
Keira Laskoski	Traceability and Documentation Manager	Records the defects and de
Dr. Mohamed Hassan	Supervisor and Stakeholder Representative	Technical expert and advis

Table 1: Verification and Validation Team Roles

- **Structured Review with Supervisor:** A structured meeting with the supervisor will occur. The SRS document will be shared with the supervisor and requirements will be reviewed. The team will prepare questions in advance, specifically focusing on the

- **Ad Hoc Peer Review and Inspection:** Classmates and primary reviewers will be recruited to perform an adhoc review of the SRS document. All criticisms will be documented as issues and addressed by a member of the team. A minimum of 5 issues created based on reviews from peers will be the objective.

- **Issue Tracker for Traceability:** As stated above, a Github issue tracker will be used to trace criticisms and suggestions for the amilioration of the SRS document. All issues will be reviewed, changes to the document will be implemented if necessary. All issues will addressed with a comment and closed.

- **Checklist-Based Inspection:** The team will prepare a checklist for necessary characteristics for a high quality issue. All requirements will be reviewed by team members to ensure each possesses these characteristics.

- **Consistency and Traceability Check:** The team will check that the use cases agree with the functional requirements, and that every functional requirement appears in the traceability tables. For example, the use case for accepting an inference request must describe the same staking behaviour as the staking and penalty requirements. The team will also check that requirement numbers in the SRS match the numbers used in this plan after each revision of the SRS.

3.3 Design Verification

- **Peer Review and Inspection:** Classmates and primary reviewers will be recruited to perform an adhoc review of the first edition of the design document after the course-imposed deadline. All criticisms will be documented as issues and addressed by a member of the team. A minimum of 5 issues created based on reviews from peers and primary reviewers will be the objective.

- **Team Member Review:** All team members will review the design document to ensure the design meets the requirements, is feasible, and is properly decomposed.

- **Checklist-Based:** The team will check the design documents against a checklist. The checklist will confirm that each functional and non-functional requirement is assigned to at least one module, that each module's interface provides the access programs needed to satisfy its requirements, and that each on-chain instruction checks the signatures and balances required by the safety requirements (FR.SA.1 to FR.SA.10).

3.4 Verification and Validation Plan Verification

- **Structured Review with Supervisor:** A structured review of the VnV plan will be performed with the supervisor, once completed. Team members will describe key components of the plan and the test environments they rely on, such as Solana Devnet and the simulation harness, and ask the supervisor whether the tests check the behaviour that matters most to requesters and providers. Feedback will be recorded as GitHub issues.

- **Peer Review and Inspection:** Classmates and primary reviewers will be recruited to perform an ad hoc review of the VnV plan. All criticisms will be documented as issues and addressed by a member of the team.

- **Team Member Review:** Each team member will review the tests in a section they did not write, checking that the expected outputs follow from the requirements and that each test can be run as described. Tests

that rely on Solana Devnet will be reviewed for timing assumptions, such as acceptance periods and prediction times.

- **Checklist-Based Inspection:** The team will check that every functional and non-functional requirement in the SRS is named in the test case derivation of at least one test, and record the result in the traceability tables. Requirements that are only partly tested will be listed so that the missing cases can be added.

3.5 Implementation Verification

- **Unit and System Tests:** All functional and non-functional requirements will be tested according to the described system tests (Section 4) and unit tests (Section 5) of this document.
- **Static Analysis and Linters:** The team will enforce code standards using linters and static analysers.
- **Code Review:** Every change will be merged into `main` through a pull request reviewed and approved by at least one other team member, implemented through CI/CD pipeline on Github. Reviewers will check the coding standards that linters cannot, such as documentation comments, and that the change includes tests.

3.6 Automated Testing and Verification Tools

The team will use the following tools. The coding standards and the tools that enforce them are set out in the Development Plan.

- **Anchor and Solana:** Used to test the on-chain programs. The tests set up accounts, run each instruction, and check account state before and after the transaction.
- **Solana Devnet:** Used for the system tests that need a real network, with test wallets funded with Devnet SOL.
- **Vitest:** Used for the unit tests of the React frontend and other TypeScript code.

- **Automated Browser Tests:** Used to run the main user flows in Chrome, Firefox, and Edge, signing transactions with a test wallet signer. Safari is tested by hand.
- **Linters and Formatters:** ESLint for TypeScript, and `cargo clippy` for Rust.
- **Continuous Integration (CI):** Github actions are implemented to run automated tests, linters, and builds on every pull request. Branch protection on `main` prevents merging while any check fails.
- **Code Coverage:** Vitest's coverage report for TypeScript and `cargo llvm-cov` for Rust. The team aims for at least 80% line coverage of the main logic modules.

3.7 Software Validation

Validation checks that DAIIM meets the needs of its requesters and providers, not only that it matches the SRS. There is no external data set for validating DAIIM, so validation relies on the supervisor, on users, and on an independent reference implementation of the aggregation algorithm.

- **Supervisor Demonstration:** The team will demonstrate the system to Dr. Hassan shortly after the scheduled Rev 0 demo. The demonstration will follow a requester and a group of providers through a complete inference request on Solana Devnet, including at least one failure case, such as a request cancelled for too few providers. The supervisor's feedback will be recorded as GitHub issues, and any change to the requirements will be made in the SRS.
- **Result Validation:** The simulation in test NFR-SIM-1 will compare every aggregated result with a reference implementation of the aggregation algorithm written by a team member who did not write the system's version. The worked examples in the SRS will be included, so that the results can also be checked by hand.
- **Requirements Review after Validation:** After the Rev-0 demonstration, the team will review the SRS with the supervisor, as in the SRS verification above, to confirm that the requirements still describe what requesters and providers need, and update the SRS and this plan if they do not.

4 System Tests

This section describes the tests that will be performed to verify that the entire integrated system meets the functional and non-functional requirements specified in the SRS. The system test cases below are be descrbed by the following characteristics:

- Control: Is this test manual or automatic?
- Initial State: What is the state of the system before the test is performed?
- Input: What is the input to the system for this test?
- Output: What is the expected output of the system for this test?
- Test Case Derivation: What FRs or NFRs does this test case cover?
- How test will be performed: What is the abstract procedure for performing the test?

4.1 Tests for Functional Requirements

4.1.1 Accounts and Authentication

These tests verify the account authentication requirements in the SRS, including authentication with a compatible Web3 wallet extension and an embedded wallet.

Authentication Using a Web3 Wallet Extension

1. ST-AUTH-EXT-1

   Control: Manual

   Initial State: The application is running in a supported browser on its public page. A compatible Web3 wallet extension is installed and active. Test profiles `EXT-USER-01` and `EXT-USER-02` are associated with wallet fixtures `EXT-WALLET-01` and `EXT-WALLET-02`; neither user is signed in.

   Input:

- Connect `EXT-WALLET-01` and approve the verification-message signature requested by the application.
- Connect `EXT-WALLET-02` and approve its verification-message signature.
- Connect `EXT-WALLET-01` and reject the signature request.

Output:

- For the approved signature from `EXT-WALLET-01`, the user is authenticated as `EXT-USER-01`, and that user's profile and authenticated features are available.
- For the approved signature from `EXT-WALLET-02`, the user is authenticated as `EXT-USER-02`, and that user's profile and authenticated features are available.
- The user remains unauthenticated, no profile or authenticated features become available.

Test Case Derivation: FR1 requires the authentication response to verify ownership of the connected wallet address.

How test will be performed: For each wallet, start from a signed-out session, connect the specified extension account, and approve the verification-message signature request. Confirm that `EXT-WALLET-01` opens `EXT-USER-01`'s profile and `EXT-WALLET-02` opens `EXT-USER-02`'s profile. Confirm that the user is granted access to the authenticated features. Repeat the process and reject the signature request; confirm that the user remains signed out and cannot access authenticated features.

2. ST-AUTH-EXT-2

   Control: Manual

   Initial State: The application is running on its public page in a supported browser. `EXT-WALLET-03` is available in a compatible browser extension and is not yet associated with a DAIIM user profile. The wallet owner is not signed in.

   Input:

- Connect `EXT-WALLET-03` and approve the verification-message signature.
- Sign out of the resulting account.
- Reconnect `EXT-WALLET-03` and approve the verification-message signature again.

Output:

- After the first approved signature, the user is authenticated and a profile is associated with `EXT-WALLET-03`. The system grants the user access to authenticated features.
- Sign-out ends the authenticated session; the user cannot access the profile or authenticated features while signed out.
- After the second approved signature, the same profile associated with `EXT-WALLET-03` is displayed. No second profile is created for the wallet. The system grants the user access to authenticated features.

Test Case Derivation: FR1 requires the system to verify ownership of the wallet address and associate that verified address with the same user profile on every successful authentication, including after sign-out. The second successful authentication must restore the profile created during the first sign-in.

How test will be performed: Record the number of DAIIM profiles associated with `EXT-WALLET-03`. Connect the wallet and approve the verification-message signature. Record the displayed profile identifier and confirm that the wallet is associated with that profile. Sign out and verify that the profile and authenticated features are no longer accessible. Sign in again with the same wallet and approve the signature. Verify that the displayed profile identifier matches the recorded identifier and that the number of profiles associated with the wallet has not increased.

Authentication Using an Embedded Wallet

1. ST-AUTH-EMB-1

Control: Automatic

Initial State: The application is running in a test environment with the embedded-wallet provider mocked. Identities `new.email@example.test`, `+15555550101`, and social-account subject `social-new-01` have no associated DAIIM account or embedded wallet. Configure the provider mock to return successful authentication and wallet creation for these identities, with distinct addresses `EMB-WALLET-NEW-01`, `EMB-WALLET-NEW-02`, and `EMB-WALLET-NEW-03`, respectively. The malformed email `not-an-email` should be rejected by the system before the provider is called.

Input:

- Submit `new.email@example.test` using the email sign-in method.
- Submit `+15555550101` using the phone sign-in method.
- Submit social-account subject `social-new-01` using the supported social sign-in method.
- Submit malformed email `not-an-email` using the email sign-in method.

Output:

- For each of the three supported identities, the system creates one embedded wallet associated with the user's account, displays its corresponding address and a creation confirmation, authenticates the user, and makes the associated profile available.
- For `not-an-email`, the system reports an input error, displays no wallet-creation confirmation or address, and does not authenticate the user or create a wallet.

Test Case Derivation: FR2 requires successful creation for supported identity input after authentication, with the public address and confirmation as outputs. The system must reject malformed identity input before wallet creation. FR1 requires the successful wallet authentication to identify the user.

How test will be performed: For each valid test identity, call the corresponding embedded-wallet sign-in method with the specified email address, phone number, or social-account identity. For each successful case, compare the displayed address with the configured mock address,

verify that the corresponding profile is authenticated, and confirm that exactly one wallet was created. For the malformed email, enter it using the email sign-in method and verify that the system reports an input error, does not call the provider, and does not create a wallet or authenticated session.

2. ST-AUTH-EMB-2

   Control: Automatic

   Initial State: The application is running in a test environment, and existing accounts `EMB-RETURN-01`, `EMB-RETURN-02`, and `EMB-RETURN-03` are signed out. Their embedded wallet addresses are respectively `EMB-WALLET-RETURN-01`, `EMB-WALLET-RETURN-02`, and `EMB-WALLET-RETURN-03`. Configure the embedded-wallet provider mock to return successful authentication responses with those address mappings. Configure it to return an authentication failure for `locked.user@example.test`, which has no associated account.

   Input:

   - Enter `returning.email@example.test` using the email sign-in method.
   - Enter `+15555550102` using the phone sign-in method.
   - Provide social-account subject `social-return-03` using the supported social sign-in method.
   - Enter `locked.user@example.test` using the email sign-in method.

   Output:

   - For `returning.email@example.test`, the system authenticates `EMB-RETURN-01` and displays its existing profile and `EMB-WALLET-RETURN-01`.
   - For `+15555550102`, the system authenticates `EMB-RETURN-02` and displays its existing profile and `EMB-WALLET-RETURN-02`.
   - For social-account subject `social-return-03`, the system authenticates `EMB-RETURN-03` and displays its existing profile and `EMB-WALLET-RETURN-03`.
   - For `locked.user@example.test`, the system reports authentication failure and does not create an authenticated session or grant access to a profile.

Test Case Derivation: FR1 requires each verified wallet address to remain associated with the same user profile after sign-out. Each successful response must restore the mapped existing profile rather than create another account. The failed authentication response does not verify wallet ownership, so it must not grant access.

How test will be performed: Call the application's embedded-wallet authentication entry point with the sign-in method and parameters from each returning-user credentials. Configure the provider mock to return that fixture's existing wallet authentication response. Assert that each call returns an authenticated session for the user profile. Then call the same entry point with the sign-in method and identity parameters from the authentication-failure credentials. Assert that the call returns no authenticated session, grants no profile access and displays the appropriate error message.

4.1.2 Inference Task Creation

This test verifies the end-to-end task creation flow, including validation of the task definition, user authorization, and confirmation that the task-creation program instruction succeeds on the blockchain.

Create and Validate Inference Tasks

1. ST-TASK-CREATE-1

   Control: Automatic

   Initial State: The system is running on Solana Devnet. The requester wallet address fixture is authenticated, and the test harness is configured with its wallet signer and sufficient test SOL to pay transaction fees. The unique task names `Regression-Test-Task`, `Regression-Test-Min-Providers`, `Regression-Test-Invalid-Schema`, and `Rejected-Authorization-Task` are not already in use.

   Input:

   - A valid numerical regression task definition named `Regression-Test-Task`, including a description, numerical input and output schemas, an

evaluation metric, a minimum provider count of 2 or more, reputation tolerance, acceptance period, and maximum prediction time.

- The valid definition named `Regression-Test-Min-Providers`, with its minimum provider count changed to 1.
- The valid definition named `Regression-Test-Invalid-Schema`, with an input or output schema containing non-numerical values incompatible with a numerical regression task.
- The valid definition named `Rejected-Authorization-Task`.

Output:

- For the valid definition with authorization approved, the task is created with the submitted definition, and the corresponding Solana Devnet transaction succeeds with the task-creation instruction.
- For the definition with fewer than 2 minimum providers, the system rejects the definition, and no task is created.
- For the definition with a schema incompatible with numerical regression, the system rejects the definition, and no task is created.
- For the valid definition with authorization rejected, no task is created and no successful task-creation transaction occurs.

Test Case Derivation: FR6 requires a task name, description, type, input schema, and output schema; the incompatible-schema case checks that the definition matches the numerical regression task. FR8 requires at least 2 providers, so a value of 1 must be rejected. FR5 permits authenticated users to create tasks, while FR4 requires wallet approval before an on-chain state change. Therefore, the valid task should be created only after approval, and rejecting authorization should leave it uncreated. FR50 and FR51 require a final transaction status and a transaction identifier that can be used to verify the successful instruction.

How test will be performed: Submit the valid definition to the task-creation flow and assert that the confirmation details match the submitted definition. Configure the test wallet signer to approve the

authorization request. After confirmation, query Solana Devnet using the returned transaction identifier and assert that the transaction succeeded and contains the task-creation instruction. Query the application to confirm that `Regression-Test-Task` is available with the submitted definition. Submit the minimum-provider and invalid-schema definitions; assert that each is rejected before authorization and no task is created. Finally, submit the valid definition named `Rejected-Authorization-Task`, configure the signer to reject authorization, and assert that no task-creation instruction is submitted and no task is created.

4.1.3 Provider Registration and Staking

These tests verify the provider registration and staking requirements in the SRS. They cover registering as an inference provider for a task, locking the required minimum stake in escrow, unregistering when eligible, and viewing provider information through the dashboard.

Registering as an Inference Provider

1. ST-PROV-1

   Control: Manual

   Initial State: The application is running in a supported browser. A provider account is authenticated and connected to a compatible wallet. The provider is not registered for inference task `TASK-01`. The task requires a minimum stake of tokens, and the provider's wallet contains sufficient tokens and SOL to complete the transaction.

   Input:

   - Navigate to `TASK-01` and select the option to register as an inference provider.
   - Review the required minimum stake of tokens and the associated registration details.
   - Confirm the registration and approve the wallet transaction.

   Output:

- The provider is successfully registered for `TASK-01`.
- Exactly the minimum amount of tokens are locked in escrow as the provider's stake.
- The provider is added to the task's list of registered providers and receives the base reputation score for the task.
- The system displays the transaction status and, once confirmed, the transaction identifier.

Test Case Derivation: FR_InferenceTaskRegistration requires authenticated users to be able to register as inference providers by staking the required minimum amount of tokens. FR_InferenceTaskRegistrationStaking requires the minimum stake to be locked in escrow. The transaction authorization and confirmation behaviour is also covered by the related transaction requirements.

How test will be performed: Authenticate a provider account and navigate to `TASK-01`. Select the registration option and verify that the system displays the required minimum stake. Confirm the registration and approve the transaction through the connected wallet. After the transaction is confirmed, inspect the task's registered-provider list and the provider's account information. Verify that the provider is registered, has received the base reputation score, and that exactly the minimum amount of tokens have been transferred into escrow.

2. ST-PROV-2

   Control: Manual

   Initial State: The application is running in a supported browser. A provider account is authenticated and connected to a compatible wallet. The provider is not registered for `TASK-02`. The task requires a minimum stake of tokens, but the provider's wallet contains fewer than 100 tokens.

   Input:

   - Navigate to `TASK-02` and select the option to register as an inference provider.
   - Attempt to proceed with registration despite having an insufficient token balance.

Output:

- The system prevents the provider from completing registration.
- An error message indicates that the provider's token balance is insufficient and identifies the required shortfall.
- No stake is transferred into escrow, and the provider is not added to the task's list of registered providers.

Test Case Derivation: FR_InferenceTaskRegistration and FR_InferenceTaskRegistrationStaki require the provider to stake the required minimum amount of tokens to register. The related balance-check assumptions require the system to verify that sufficient funds are available before processing the transaction.

How test will be performed: Authenticate a provider account with fewer than 100 tokens and navigate to `TASK-02`. Attempt to register for the task. Verify that the system displays an insufficient-balance error, that no transaction transfers tokens into escrow, and that the provider does not appear in the task's registered-provider list.

Unregistering from an Inference Task

1. ST-PROV-3

   Control: Manual

   Initial State: The application is running in a supported browser. A provider account is authenticated and registered for `TASK-03`. The provider has no active accepted inference requests for this task. The provider's current reputation score is at or above the task's reduced-stake-return threshold, and the provider has a remaining stake of tokens.

   Input:

   - Navigate to the provider's registration details for `TASK-03`.
   - Select the option to unregister from the task.
   - Review and confirm the unregistration.
   - Approve the wallet transaction, if prompted.

Output:

- The provider is successfully unregistered from `TASK-03`.
- The provider is removed from the task's list of registered providers.
- The provider's entire remaining stake of tokens is returned to their token account.
- The system displays the transaction status and, once confirmed, the transaction identifier.

Test Case Derivation: FR_ProviderWillingUnregistration requires the system to allow providers to unregister when they have no active accepted inference requests. FR_ProviderUnregistrationStakeReturn requires the entire remaining stake to be returned when the provider's reputation is at or above the task's reduced-stake-return threshold.

How test will be performed: Authenticate the registered provider and verify that they have no active accepted inference requests for `TASK-03`. Select the unregistration option and confirm the action. After the transaction is confirmed, inspect the provider's registration status, the task's registered-provider list, and the provider's token balance. Verify that the provider has been removed from the list and that the entire remaining stake has been returned.

2. ST-PROV-4

   Control: Manual

   Initial State: The application is running in a supported browser. A provider account is authenticated and registered for `TASK-04`. The provider has accepted an inference request that is still active for this task.

   Input:

   - Navigate to the provider's registration details for `TASK-04`.
   - Select the option to unregister from the task.
   - Attempt to confirm the unregistration.

   Output:

- The system rejects the unregistration attempt because the provider has an active accepted inference request.
- An error message explains why the provider cannot unregister.
- The provider remains registered for `TASK-04`, and their remaining stake is unchanged.

Test Case Derivation: FR_ProviderWillingUnregistration requires that a provider must not have accepted any active inference requests for the task before they can unregister.

How test will be performed: Authenticate a provider account that is registered for `TASK-04` and has an active accepted inference request. Attempt to unregister from the task. Verify that the system rejects the attempt, displays an appropriate error message, maintains the provider's registration, and does not modify the provider's remaining stake.

Viewing the Provider Dashboard

1. ST-PROV-5

   Control: Manual

   Initial State: The application is running in a supported browser. A provider account is authenticated and registered for two inference tasks, `TASK-05` and `TASK-06`. The provider has an existing reputation score, staked token balance, and earned rewards recorded for each task.

   Input:

   - Navigate to the provider dashboard.
   - View the provider's information for `TASK-05` and `TASK-06`.

   Output:

   - The dashboard displays the provider's reputation score, staked tokens, and earned rewards for `TASK-05`.
   - The dashboard displays the provider's reputation score, staked tokens, and earned rewards for `TASK-06`.
   - The displayed values correspond to the provider's recorded information for each task.

Test Case Derivation: FR_ProviderDashboard requires inference providers to be able to view their reputation score, staked tokens, and rewards earned for each inference task they are registered for.

How test will be performed: Authenticate the provider account and navigate to the provider dashboard. Retrieve the provider's recorded reputation, stake, and reward values for both tasks from the system's records. Compare these values with the information displayed on the dashboard and verify that the correct information is displayed for each task.

4.1.4 Inference Requests

These tests verify the inference request requirements in the SRS. They cover submitting requests for existing inference tasks, validating request input data against the task's input schema, authorizing payments into escrow, and broadcasting submitted requests to registered providers.

Submitting an Inference Request

1. ST-REQ-1

   Control: Manual

   Initial State: The application is running in a supported browser. A requester account is authenticated and connected to a compatible wallet. An existing inference task, `TASK-01`, is available and has at least two registered providers. The task's input schema requires a numerical input field named `input_value`. The requester's wallet contains sufficient tokens and SOL to submit the request.

   Input:

   - Navigate to `TASK-01` and select the option to submit an inference request.
   - Provide valid input data containing `input_value` with a numerical value
   - Review the request details and confirm the submission.
   - Authorize the payment through the connected wallet.

Output:

- The system accepts the input data and successfully submits the inference request for `TASK-01`.
- The request is assigned a unique identifier and its submission status is displayed to the requester.
- The input data is stored securely for processing and is not recorded directly on-chain.
- The requester is informed that the request has been submitted and is awaiting provider participation.

Test Case Derivation: FR_RequestSubmission requires authenticated users to be able to submit inference requests for existing tasks, with input data conforming to the task's input schema. The related input-data requirements specify that input data must be stored securely and not placed directly on-chain.

How test will be performed: Authenticate a requester account and navigate to `TASK-01`. Enter the valid input data and submit the request. Authorize the transaction through the connected wallet. Verify that the system creates a request with a unique identifier, displays the appropriate submission status, and securely stores the input data. Inspect the on-chain request record to confirm that the input data itself has not been recorded on-chain.

2. ST-REQ-2

   Control: Manual

   Initial State: The application is running in a supported browser. A requester account is authenticated and connected to a compatible wallet. An existing inference task, `TASK-02`, is available and has at least two registered providers. The task's input schema requires a numerical field named `input_value`.

   Input:

   - Navigate to `TASK-02` and select the option to submit an inference request.

- Provide input data with the required `input_value` field missing.
- Repeat the submission attempt with `input_value` provided as a non-numerical value.

Output:

- The system rejects both invalid input submissions.
- The system displays an error message identifying the missing or invalid input field.
- No inference request is submitted, no request identifier is created, and no payment is transferred into escrow.

Test Case Derivation: FR_RequestValidation requires the system to reject inference requests whose input data does not conform to the task's input schema. FR_RequestSubmission requires submitted requests to include input data that conforms to that schema.

How test will be performed: Authenticate a requester account and navigate to `TASK-02`. Attempt to submit a request with the required input field missing. Verify that the system rejects the request and identifies the missing field. Repeat the test using a non-numerical value for `input_value`. Verify that the request is rejected again and that neither attempt creates a request or transfers funds into escrow.

Request Payment and Authorization

1. ST-REQ-3

   Control: Manual

   Initial State: The application is running in a supported browser. A requester account is authenticated and connected to a compatible wallet. An existing inference task, `TASK-03`, is available and has at least two registered providers. The task has a minimum payment requirement. The requester's wallet contains sufficient tokens and SOL to authorize a payment

   Input:

   - Navigate to `TASK-03` and provide valid input data.

- Select a payment amount of tokens.
- Review the transaction details and confirm the request submission.
- Approve the transaction through the connected wallet.

Output:

- The system accepts the payment amount because it exceeds the task's minimum payment requirement.
- The wallet prompts the requester to authorize a transaction transferring the tokens into escrow.
- After authorization and transaction confirmation, exactly those amount of tokens are transferred into escrow.
- The system displays the transaction status and, once confirmed, the transaction identifier.
- The amount transferred into escrow matches the amount authorized by the requester.

Test Case Derivation: FR_RequestPayment requires the requester to choose a token payment greater than the minimum and authorize a wallet transaction transferring that amount into escrow. The related transaction authorization and preview requirements ensure that the requester can review and authorize the transaction. SA_ChargeEqualsAuthorized requires the amount charged to match the amount authorized.

How test will be performed: Authenticate a requester account and navigate to `TASK-03`. Enter valid request input and select a payment amount for a set amount of tokens. Review the transaction preview and approve the transaction through the connected wallet. After confirmation, inspect the requester's token balance and the escrow account's balance. Verify that exactly the set amount of tokens have been transferred into escrow and that the amount matches the requester's authorization.

Broadcasting Submitted Inference Requests

1. ST-REQ-4

Control: Automatic

Initial State: The system is running in a test environment. An existing inference task, `TASK-04`, has three registered providers: `PROV-01`, `PROV-02`, and `PROV-03`. All three providers are available to receive inference requests. The task has a configured acceptance period and maximum prediction time.

Input:

- Submit a valid inference request for `TASK-04`.
- Set the proposed payment amount to a number of tokens.
- Configure the acceptance-period expiry time to 10 minutes after submission.
- Configure the maximum prediction time to 30 minutes.

Output:

- The system successfully submits the inference request.
- The system broadcasts the request to all three registered providers.
- Each provider receives the task definition, proposed payment amount, acceptance-period expiry time, and maximum prediction time.
- The broadcast information is consistent across all three providers and matches the submitted request.

Test Case Derivation: FR_RequestBroadcast requires the system to broadcast the task definition, proposed payment amount, acceptance-period expiry time, and maximum prediction time to all providers registered for the task after an inference request is submitted.

How test will be performed: Configure `TASK-04` with the three specified registered providers and submit a valid inference request with the specified payment amount and timing values. Monitor the messages or notifications received by each provider. Verify that all three providers receive the broadcast and that the task definition, proposed payment, acceptance-period expiry time, and maximum prediction time match the submitted request.

4.1.5 Reputation, Rewards, and Penalties

These tests verify the reputation, reward, and penalty requirements (FR35–FR48) and the safety requirements that protect reward and penalty transfers (FR.SA.9, FR.SA.10). Each test follows a scenario that a group of providers would go through on the real system: completing a request, misbehaving during a request, and leaving a task. The tests run on a local Solana validator or Solana Devnet, with mock inference providers that submit preset predictions, so that the expected results can be worked out by hand before the test is run. Boundary cases for each rule (such as a distance exactly equal to $\bar{d}$) are covered by the unit tests.

The values of `SLASH_FRACTION`, `MAX_REPUTATION_PENALTY`, `MINIMUM_PROVIDER_STAKE`, `REPEAT_OFFENSE_COUNT`, and `REPEAT_OFFENSE_WINDOW` are defined in the SRS. The tests read them from the deployed program's configuration, so they do not need to change if the values change.

1. ST-REP-1: Completed request updates reputations and pays rewards

   Control: Automatic

   Initial State: A numerical regression task `Rep-Test-Task` exists with a reputation tolerance $T = 2.0$. Mock providers `P1` to `P5` are registered with reputation scores of 40, 60, 50, 50, and 50. A requester has submitted an inference request with a payment of 1000 tokens held in escrow, and all five providers have accepted it.

   Input: The providers submit the valid predictions `P1`: 8.5, `P2`: 11, `P3`: 12, `P4`: 13, `P5`: 20, and the request runs to completion.

   Output:

   - The final aggregated result is 12. The distances from the result are $d = (3.5, 1, 0, 1, 8)$, so $\bar{d} = 2.7$ and $\bar{d} + T = 4.7$.
   - `P2`, `P3`, and `P4` ($d_i \leq 2.7$) have their reputation increased. `P1` ($2.7 < 3.5 \leq 4.7$) has an unchanged reputation. `P5` ($8 > 4.7$) has its reputation decreased.
   - `P5` receives no reward. `P1` to `P4` share the 1000 tokens in proportion to their reputation scores (40, 60, 50, 50): 200, 300, 250, and 250 tokens.

- The total paid out does not exceed 1000 tokens, the request's escrow is empty afterwards, and the system logs a status check for each provider before its reward is sent.

Test Case Derivation: FR35 requires d_i and $\bar{d}$ to be computed for every included provider, and FR37, FR38, and FR39 give the three possible outcomes; this fixture places at least one provider in each. FR41 limits rewards to providers whose reputation did not decrease, split by reputation, so `P1` still qualifies and `P5` does not. FR.SA.9 forbids paying out more than the reward, and FR.SA.10 requires each provider's status to be checked before a reward is sent. The expected reward shares use the reputation snapshot taken for aggregation (FR31); if the design uses the scores after the update instead, the expected shares will be recomputed from those scores.

How test will be performed: Record each provider's reputation and token balance, run the request, and read the result, the reputation scores, and the reward transfers from both the application and the on-chain accounts. Assert the values above, allowing up to one base unit of difference per reward for integer rounding, and check the backend log for the status checks.

2. ST-PEN-1: Misbehaving providers in a single request

   Control: Automatic

   Initial State: Task `Penalty-Test-Task` has a numerical output schema. Providers `P1` to `P5` are registered with 100 tokens of stake each. `P1` has a reputation score of 100, and the others have 50. A request has been submitted and all five providers have accepted it.

   Input:

   - `P1` and `P2` submit accurate predictions (10 and 10.5).
   - `P3` submits a valid but very inaccurate prediction (1,000,000).
   - `P4` submits a prediction that does not match the output schema (a string instead of a number).
   - `P5` submits nothing, and the maximum prediction time is allowed to pass.

Output:

- P1's reputation is increased but stays at 100.
- P3's reputation decreases by no more than MAX_REPUTATION_PENALTY, and its stake stays at 100 tokens.
- P4's prediction is rejected. P4 and P5 each have their stake reduced to $100 \times (1 - \texttt{SLASH_FRACTION})$ tokens and their reputation decreased.
- P2's stake is unchanged, and every reputation score stays within $[0, 100]$.

Test Case Derivation: FR43 requires slashing and a reputation decrease for a rejected prediction, and FR44 requires the same for a provider that accepted but did not submit. FR42 allows slashing only in those two cases, so the inaccurate but valid prediction from P3 must cost reputation and not stake. FR40 caps the reputation decrease from a single request, which the extreme prediction from P3 checks. FR36 keeps scores within $[0, 100]$, which P1 checks at the top of the range.

How test will be performed: Use a task with a short maximum prediction time. Record each provider's stake and reputation, run the request until it completes, and read the values again from the on-chain stake and reputation accounts. Assert the expected values for each provider.

3. ST-PEN-2: Providers leaving a task

   Control: Automatic

   Initial State: Task Exit-Test-Task has an exit reputation threshold $R_{\text{exit}} = 50$. The following providers are registered:

   - P1: stake just above MINIMUM_PROVIDER_STAKE, so that one slash drops it below the minimum.
   - P2: a large stake, so that slashing alone will not drop it below the minimum during the test.
   - P3: a reputation score low enough that one decrease brings it to 0, and a stake of 100 tokens.
   - P4, P5, and P6: 100 tokens of stake each and reputation scores of 80, 50, and 25.

Input:

- Run REPEAT_OFFENSE_WINDOW requests for the task. P1 fails to submit in the first one. P2 fails to submit in REPEAT_OFFENSE_COUNT of them. P3 submits a very inaccurate prediction in the first one. The other providers submit accurate predictions.
- After the requests finish, P4, P5, and P6 voluntarily unregister.
- While one of the requests is active, P4 tries to unregister.

Output:

- P1 is unregistered automatically after its stake falls below MINIMUM_PROVIDER_STAKE.
- P2 is unregistered after its REPEAT_OFFENSE_COUNT-th slash, and its remaining stake is burned.
- P3's reputation reaches 0. P3 is unregistered and its remaining stake is burned.
- P4's attempt to unregister during an active request is refused. After the requests, P4 ($R = 80$) and P5 ($R = 50 = R_{\text{exit}}$) get all 100 tokens back. P6 ($R = 25$) gets $\frac{25}{50} \times 100 = 50$ tokens back, and the other 50 are burned.

Test Case Derivation: FR45, FR46, and FR47 give the three cases where a provider is removed automatically: stake below the minimum, too many slashes, and a reputation of 0. FR46 and FR47 also require the remaining stake to be burned. FR48 sets how much stake is returned on voluntary unregistration, and P5 sits exactly on the threshold. The refused attempt by P4 also exercises FR15.

How test will be performed: After each request, record each provider's registration status, stake, and slash count. Record the total burned token supply and each provider's token balance before and after unregistration, and assert the expected changes.

4.1.6 Transparency and Data Handling

These tests verify that users can see and verify the transactions they authorize (FR49–FR52) and that a request's input data is protected for its whole life (FR53–FR55). They follow a requester who pays for a request and checks it, and a request's input data from submission to deletion.

1. ST-TRANS-1: Requester pays for a request and verifies it

   Control: Automatic

   Initial State: An authenticated requester is signed in with a test wallet signer that holds 50 tokens. Task `Rep-Test-Task` and the providers from ST-REP-1 are registered.

   Input:

   - Submit an inference request with a payment of 137 tokens.
   - Add funds so the wallet holds 1000 tokens, then submit the request again and approve the transaction.
   - Let the request run to completion, then open the transaction history.

   Output:

   - On the first attempt, the system shows an insufficient-funds error and does not ask the wallet for a signature.
   - On the second attempt, before the wallet asks for a signature, the application shows the action ("Submit inference request") and the amount (137 tokens).
   - The transaction status is shown as pending and then confirmed.
   - The requester's token balance goes down by exactly 137 tokens, and the request's escrow holds exactly 137 tokens.
   - The transaction history shows an identifier for the payment. Looking it up on Solana Devnet returns a transaction signed by the requester's wallet with the expected instruction and amount.
   - The blockchain holds records of the payment, the providers' stakes, the aggregated result, the reputation updates, and the reward transfers, and each matches the value shown in the application.

   Test Case Derivation: FR.SA.2 requires a balance check before a signature is requested. FR49 requires the action and amount to be shown before signing, and FR50 requires the status to be shown until it is final. FR.SA.8 requires the amount charged to equal the amount approved; an unusual amount (137) is used so that a hard-coded or rounded value

would be caught. FR51 requires an identifier that can be looked up on the blockchain, and FR52 lists the records that must be on the blockchain.

How test will be performed: Run the flow in an automated browser test. Pause the test wallet signer when it receives a signing request and check the confirmation screen. Record each status shown. Compare balances before and after. Query Solana Devnet with the transaction identifier and read the request, stake, and reputation accounts directly, without going through the application's backend.

2. ST-DATA-1: Input data from submission to deletion

   Control: Automatic

   Initial State: Task `Data-Test-Task` has registered providers `P1`, `P2`, and `P3`. Another authenticated user `U1` is not involved.

   Input:

   - Requester `R1` submits request `REQ-A` whose input contains the marker value `271828.1828`. `P1` and `P2` accept it; `P3` does not.
   - During the prediction phase, `R1`, `P1`, `P2`, `P3`, `U1`, and an unauthenticated client each try to get the input data.
   - `REQ-A` runs to completion.
   - `R1` submits request `REQ-B`, which is cancelled because too few providers accept it.

   Output:

   - Only `R1`, `P1`, and `P2` can get `REQ-A`'s input. `P3`, `U1`, and the unauthenticated client are refused.
   - No transaction or account linked to either request contains the marker value in readable form.
   - After each request reaches its final state, its input is no longer in any storage the system controls, and every user, including `R1`, gets "not found" when asking for it.

Test Case Derivation: FR53 limits the input to the requester and the providers that accepted; `P3` checks that being registered is not enough. FR54 forbids readable input on the blockchain, and the marker value makes it possible to search raw on-chain data for it. FR55 requires deletion after both completion and cancellation. The test also exercises FR24 (input given to accepting providers) and FR23 (cancellation when too few providers accept).

How test will be performed: Call the input-data endpoint as each user. Search the raw data of every linked transaction and account for the marker in its common encodings (text, 32-bit and 64-bit floating point, and the program's fixed-point format). After each request ends, query the database, file storage, and caches directly and through the application.

4.1.7 Safety

This test verifies the transaction safety requirements that are not already covered by the tests above (FR.SA.1 and FR.SA.3–FR.SA.7). FR.SA.2 and FR.SA.8 are covered by ST-TRANS-1, and FR.SA.9 and FR.SA.10 by ST-REP-1. The test injects faults using a test build of the backend and a proxy between the system and the Solana RPC.

1. ST-SAFE-1: Requester payments under transaction faults

   Control: Automatic, with the retry step also checked manually in the web client

   Initial State: A requester is signed in with 1000 tokens. The RPC proxy and the backend test build can each be set to inject one fault.

   Input: The requester submits an inference request payment several times, with a different fault each time:

   (a) The proxy forwards the transaction but drops the response.

   (b) The proxy holds the transaction until its blockhash has expired.

   (c) The requester's tokens are moved out of the wallet after the balance check but before the transaction runs.

(d) The requester rejects the signature, and separately a test script sends a payment signed by a different wallet directly to the program.

(e) The backend test build changes the transaction's amount after the amount has been shown to the user.

(f) After any of the failures above, the requester presses retry.

Output:

(a) The system checks the status before trying again, finds the payment was processed, and does not resubmit. Exactly one payment is made and each status is logged.

(b) The system only marks the payment as failed after the validity period has ended and it has checked the status.

(c) The on-chain program rejects the payment and no state changes.

(d) The rejected signature is never submitted, and the program rejects the payment signed by the wrong wallet.

(e) The simulation does not match the shown amount, so the transaction is not sent for signing.

(f) The user sees an error message with an option to retry, and the retry succeeds.

Test Case Derivation: Each fault matches one requirement: FR.SA.1 (no resubmission before confirming the first was not processed, with status logs), FR.SA.5 (wait for expiry and a status check), FR.SA.3 (the on-chain program checks funds at execution), FR.SA.4 (only signed transactions are submitted), FR.SA.7 (simulation must match the shown amounts), and FR.SA.6 (show the error and allow a retry). After the whole test, the requester should have paid only for the requests that succeeded.

How test will be performed: Run each fault in turn. Use the proxy to count submitted transactions and the test wallet signer to count signing requests. Count payments from the requester's wallet on Devnet and compare the final balance with the number of successful requests. A tester also runs step 6 by hand in the web client and records the error message shown.

4.2 Tests for Nonfunctional Requirements

This section describes the tests for the non-functional requirements in the SRS (NFR1–NFR17 and NFR.SA.1–NFR.SA.4). As with the functional tests, each test is a larger activity that checks several related requirements at once: a large simulation of many requests, a usability study, a security and privacy audit, and a set of release checks. Some tests produce measurements, tables, or graphs rather than a single pass or fail result. Values that have not been decided yet are written as symbolic constants and are listed in the Symbolic Parameters appendix.

4.2.1 Accuracy, Reliability, and Scalability

1. NFR-SIM-1: Large-scale request simulation

   Control: Automatic

   Initial State: The system runs on a local Solana validator with a simulation harness that controls mock providers and can inject faults. A separate reference implementation of the Appendix A aggregation algorithm is written in TypeScript by a team member who did not write the system's version.

   Input/Condition:

   - Run `NUM_SIMULATED_REQUESTS` requests with random predictions and reputation scores, including values at the edges of their declared ranges (reputation 0 and 100, the largest and smallest allowed predictions and token amounts) and the worked examples from the SRS.
   - Mix in failure scenarios: too few providers accept, too few valid predictions, providers that do not submit, invalid predictions, dropped or delayed RPC responses, and a backend restart during the prediction phase.
   - Run the requests at 1, 5, 10, 20, and `MAX_CONCURRENT_REQUESTS` at the same time. Mock providers reply immediately, so only the system's own processing time is measured.
   - During each prediction phase, try to read submitted predictions as another provider, as the requester, and from the raw on-chain

data.

Output/Result:

- Every aggregated result equals the reference implementation's result. The test reports the number of mismatches (expected: 0).
- No calculation overflows. Any input that would overflow causes the transaction to fail with no change to on-chain state.
- Every request reaches a final state before its final deadline, and its escrow is empty afterwards.
- No prediction can be read before its prediction phase ends.
- A table and graph of the median and 95th-percentile processing time at each concurrency level. The test passes if the median at `MAX_CONCURRENT_REQUESTS` is no more than `PERFORMANCE_DEGRADATION_TOLERANCE` higher than for a single request.

Test Case Derivation: NFR1 requires simulation tests to show that the results match the Appendix A algorithm. NFR2 forbids overflow and requires an overflowing transaction to leave no change. NFR13 requires every request to reach a final state and settle its escrow before the final deadline. NFR11 requires predictions to stay hidden until the phase ends. NFR15 requires concurrent requests without a drop in performance.

How test will be performed: The harness runs the requests and then reads each request's status, result, and escrow from the blockchain. It compares each result with the reference implementation and logs the inputs of any mismatch so they can be added as a unit test. It checks that the tokens escrowed at the start equal the tokens paid out, returned, slashed, and burned. Each concurrency level is run 3 times, and the timings come from the system's logs.

4.2.2 Usability

1. NFR-USE-1: Usability study with new users

Control: Manual, with timing checks automated

Initial State: The web application is deployed on a test environment connected to Solana Devnet. 5 to 8 participants who use a web browser but have never used a blockchain wallet or built a machine learning model are recruited.

Input/Condition: Each participant gets a short description of DAIIM, but no instructions on how to use it, and completes these tasks while thinking out loud:

(a) Sign in by creating an embedded wallet with an email address.

(b) Find the documentation and the list of available tasks.

(c) Submit an inference request for an existing task.

(d) Find the result of the request and the payment for it.

(e) Sign out.

During task 3, the test build forces one error (a failed transaction). The participant then fills out the survey in the Usability Survey Questions appendix.

Output/Result:

- At least `USABILITY_COMPLETION_RATE` of participants complete every task without help, including creating a wallet with no Web3 knowledge.
- The median time to finish tasks 1 to 3 is 5 minutes or less.
- Every feature can be reached within 2 clicks of the home page.
- Every error shown has a name, a description, and a suggested fix, and appears within 2 seconds of the failure being detected.
- A progress indicator is shown during every operation longer than `LONG_OPERATION_THRESHOLD`.
- A plain-language confirmation is shown after every transaction.
- The survey results are summarized in a table and bar chart.

Test Case Derivation: NFR3 requires a new user to learn the system within 5 minutes, and NFR8 requires them to create an embedded wallet without Web3 knowledge. NFR4 requires every feature within 2 clicks. NFR5 requires errors to have a name, description, and fix, and

NFR.SA.1 requires them within 2 seconds. NFR6 requires progress to be shown for long operations, and NFR7 requires plain-language confirmations.

How test will be performed: A team member observes each session, times each task, records clicks, and only helps if the participant is stuck for more than 3 minutes (which counts as not completing the task). A team member also counts the clicks to each feature from the home page. Error timings and progress indicators are checked by an automated browser test that repeats the same tasks with forced errors and delays. One study is held after the Rev 0 demo and a second after changes are made in response to it.

4.2.3 Security, Privacy, and Safety

1. NFR-SEC-1: Security and privacy audit

   Control: Manual, with automated scans

   Initial State: The system is deployed in its test environment. A request is in progress, so its input data is stored. Test providers have consented to showing different sets of profile information.

   Input/Condition:

   - Run a full request with an extension wallet and with an embedded wallet, capturing network traffic and backend logs.
   - Read the raw contents of the database and file storage.
   - Scan every endpoint's TLS setup, connect over plain HTTP, and point the backend at RPC and storage endpoints that use a self-signed certificate.
   - Visit every public page without signing in.

   Output/Result:

   - Input data is encrypted in all captured traffic and cannot be read in raw storage, using algorithms and key lengths on a recognized recommended list (for example, NIST SP 800-175B).

- All endpoints use TLS 1.2 or later with valid certificates. Plain HTTP is refused or redirected, and the backend refuses endpoints with invalid certificates.
- No private key or recovery phrase appears in traffic, logs, or storage, and no email address or phone number appears on the blockchain.
- Public pages show only the information each provider agreed to share.
- A completed audit checklist based on the PIPEDA safeguards principle, with any gaps recorded as issues.

Test Case Derivation: NFR9 and NFR10 require sensitive data to be encrypted in transit and at rest to an industry standard, verified through a security audit. NFR.SA.3 requires TLS with certificate checks. NFR.SA.2 requires all signing to happen in the user's wallet with no private information on the blockchain. NFR12 limits public pages to information providers consented to.

How test will be performed: A team member who did not write the storage code leads the audit. Search the captured traffic, logs, storage, and on-chain data for marker values in the input and for the test wallets' keys and the test users' contact details. Run `testssl.sh` against each endpoint. Compare each public page with the providers' consent settings. The team reviews the checklist in a meeting.

4.2.4 Maintainability, Portability, and Installability

1. NFR-RELEASE-1: Release checks

   Control: Automatic, with a manual run on Safari

   Initial State: A release candidate is on a pull request to `main`, and the GitHub Actions workflow is set up.

   Input/Condition:

   - CI runs `ESLint`, `Prettier --check`, `cargo clippy` (with warnings treated as errors), `cargo fmt --check`, `npm audit`, and `cargo audit`.

- The main user flow (sign in with an embedded wallet, browse tasks, submit a request, view the result and transaction history, sign out) is run on the current versions of Chrome, Firefox, Edge, and Safari, each in a new browser profile with no extensions installed.

Output/Result:

- The linters and formatters report no errors, and the pull request cannot be merged while any check fails.
- No dependency vulnerability of high or critical severity is reported unless the team has reviewed it and recorded it in an issue.
- The flow completes in every browser with no console errors and without asking the user to install any software or extension. The result is a table of each browser and step.

Test Case Derivation: NFR14 requires the coding standards in the Development Plan, enforced by linting; the Development Plan names these tools. NFR.SA.4 requires dependencies to be scanned for known vulnerabilities. NFR16 requires the system to work on current modern browsers, and NFR17 requires embedded-wallet users to need nothing beyond a browser.

How test will be performed: The linters and scans run in CI on every pull request and once a week, and branch protection on `main` requires them to pass. The browser flow runs as an automated test on Chrome, Firefox, and Edge in CI. A team member runs it by hand on Safari, since Safari cannot run in CI. Reviewers check the standards that linters cannot (such as `rustdoc` comments) during pull request review.

4.3 Traceability Between Test Cases and Requirements

5 Unit Test Description

5.1 Unit Testing Scope

5.2 Tests for Functional Requirements

5.2.1 Module 1

1. test-id1

   Type:
   Initial State:
   Input:
   Output:
   Test Case Derivation:
   How test will be performed:

2. test-id2

   Type:
   Initial State:
   Input:
   Output:
   Test Case Derivation:
   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

5.3.1 Module ?

1. test-id1

   Type:
   Initial State:
   Input/Condition:
   Output/Result:
   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.
   Initial State:
   Input:
   Output:
   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

The constants used by the system itself (`MINIMUM_PROVIDER_STAKE`, `MAX_REPUTATION_PENALTY`, `SLASH_FRACTION`, `REPEAT_OFFENSE_COUNT`, `REPEAT_OFFENSE_WINDOW`, and `USABILITY_COMPLETION_R` are defined in Appendix C of the SRS. The constants below are only used by the tests in this document.

- `LONG_OPERATION_THRESHOLD`: the length of time after which an operation counts as "long" and must show progress (NFR-USE-1). Proposed value: 1 second.
- `NUM_SIMULATED_REQUESTS`: the number of requests run in the reliability simulation (NFR-SIM-1). Proposed value: 500, spread across the listed scenarios.
- `MAX_CONCURRENT_REQUESTS`: the largest number of concurrent requests in the scalability test (NFR-SIM-1). Proposed value: 50.
- `PERFORMANCE_DEGRADATION_TOLERANCE`: the largest allowed increase in median processing time at `MAX_CONCURRENT_REQUESTS` compared with a single request (NFR-SIM-1). Proposed value: 25%.

6.2 Usability Survey Questions

Participants answer this survey at the end of each usability session described in test NFR-USE-1. Questions 1 to 8 use a scale from 1 (strongly disagree) to 5 (strongly agree).

1. It was easy to sign in and create a wallet.
2. I understood what a wallet was used for in DAIIM after signing in.
3. I could find the features I needed without searching for a long time.

4. I understood what I was paying for before I approved each payment.
5. When something went wrong, the error message told me what happened and what to do next.
6. While waiting for something to finish, I could tell that the system was still working.
7. After each payment, I understood what had happened and whether my balance had changed.
8. I would be comfortable using DAIIM again without help.
9. Which task was the hardest to complete, and why?
10. Were there any words or messages you did not understand? Which ones?
11. Was there anything you expected to be able to do but could not?
12. Do you have any other comments about the application?

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

An AI assistant using the Copilot SDK in Visual Studio Code was used to assist with revisions to this document. The specific model used is GPT-6 Luna.

7.2 How and Where Used

The AI assistant was used to discuss, draft, and revise prose and requirements for the verification and validation work. In the VnV Plan, it helped structure system tests by functional area and draft and refine test cases for wallet authentication, embedded-wallet creation, returning-user authentication, and inference-task creation. Revisions focused on making inputs, expected outputs, test procedures, and requirement derivations clearer and more specific.

The assistant also helped revise related SRS content: clarifying support for embedded and browser-extension wallets, adding requirements for embedded-wallet creation and persistent wallet-to-profile association, and adding a conservative User Profile entry to the major data entities and data dictionary. The team directed the scope and reviewed or adjusted the proposed content.

7.3 Verification of AI-Generated Content

AI-assisted content in this document was checked against the SRS to confirm that each test case addresses stated requirements and does not introduce unapproved system behaviour or design decisions. Requirement references and the expected outcomes derived from them were checked against the corresponding SRS requirements and use cases. Test cases were reviewed for clear inputs, expected results, and procedures, and the LaTeX structure and cross-references were inspected after editing. Test fixtures, schema examples, and blockchain verification details are proposed examples and must be reconciled with the implemented system before execution. The team remains responsible for the accuracy, completeness, and consistency of this document, regardless of whether material was first drafted by a team member or with the help of an AI tool.

8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

   Aidan: Splitting the system tests by SRS subsection worked well. Because our functional requirements were already grouped by the phase of an inference request, each of us could take a set of subsections and write the tests for them without waiting on anyone else. Writing tests also made the SRS easier to check. For the reputation and reward tests, I worked out small sets of predictions by hand, so that each provider's distance from the result fell into a known case (increase, no change, or decrease). Doing this showed that the reputation rules in the SRS are precise enough to test, and it gave us expected values we can reuse when we write the automated tests later.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   Aidan: The main pain point was that some parts of the SRS were not yet specific enough to test. For example, several constants such as `SLASH_FRACTION` and `MAX_REPUTATION_PENALTY` are still "to be determined", and the reward requirement does not say whether rewards are split using reputation scores from before or after the request's reputation update. To handle this, I wrote the tests in terms of the symbolic

constants so they will not need to change once the values are chosen, and I wrote down the assumption made in the reward test so the team can confirm it and update the SRS.

Another pain point was not knowing how much detail the system tests needed, or whether every requirement needed its own test. My first draft had one small test for almost every requirement, which made my sections very long and repeated the same setup many times. After comparing it with an example VnV plan from another capstone group, I realized that system tests should follow larger, realistic scenarios, like a requester paying for a request and checking it on the blockchain, or a group of providers misbehaving during one request. I rewrote my sections so that each test covers about five to seven related requirements, and left the small edge cases, such as a distance exactly equal to the average, for the unit tests. To make sure nothing was lost in the rewrite, I checked that every requirement in my sections was still named in at least one test. We also added the question of how detailed the system tests should be to our list of questions for our TA.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project?

    - **Testing Solana programs (Robert):** Writing automated tests for our on-chain programs with the Anchor test framework and a local Solana validator, including setting up accounts and checking account state before and after a transaction.
    - **Simulation and fault injection (Aidan):** Building a simulation harness with mock providers that can run many requests and inject failures, such as dropped RPC responses or providers that never submit. This is needed for the reliability, scalability, and safety tests.
    - **Usability testing (Keira):** Planning and running usability sessions, observing participants without leading them, and turning survey answers and observations into changes to the interface.
    - **Security testing (Nathan):** Checking encryption in transit and at rest, scanning TLS configurations, searching network traffic and on-chain data for leaked information, and running dependency scans.

- **Frontend and end-to-end testing (all):** Writing Vitest tests for the React frontend and automated browser tests that sign transactions with a test wallet.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

    - **Testing Solana programs:** (1) Work through the testing sections of the Anchor documentation and its example projects, or (2) read the tests of existing open-source Solana programs. Robert will start with the Anchor documentation, since he already has some Solana experience and the documentation matches the version of Anchor we use.
    - **Simulation and fault injection:** (1) Build a small prototype harness during the proof of concept and grow it, or (2) study how existing tools inject network faults, such as proxy-based tools. Aidan will build a prototype during the proof of concept, because the harness depends on our own request flow, and starting small will show early which faults are hard to set up.
    - **Usability testing:** (1) Apply the methods from SFWRENG 4HC3, or (2) run a small pilot session with a classmate before the real sessions. Keira will use the 4HC3 material, since she is taking the course now, and the team will run a pilot session to test the tasks and survey.
    - **Security testing:** (1) Follow guides such as the OWASP Web Security Testing Guide, or (2) learn from the cryptography and network security courses some members have taken. Nathan will follow the OWASP guide because it gives step-by-step tests, and he will ask Robert and Keira for help with the cryptography and network questions their courses cover.
    - **Frontend and end-to-end testing:** (1) Read the Vitest and browser-testing tool documentation, or (2) pair program the first few tests and then review each other's tests in pull requests. The team will pair program the first tests, so everyone learns the same

patterns, and then rely on pull request reviews to keep the tests consistent.